

Individual Project: Step Counting with Smartphone IMUs

Zhao Mingze

u3665791@connect.hku.hk

HKU

Hong Kong

1 Introduction

This is COMP7310 individual project report. Step counting is a core feature in mobile health apps, but designing a robust algorithm across device placements, walking styles, and real-time streaming is challenging. This project builds the sensing core of a step-counting application using smartphone IMU data, focusing on algorithmic aspects.

The goal is a unified step-counting system for offline and online streaming. The StepCounter is robust to phone orientation, uses accelerometer-only input, outputs accurate step counts and timestamps, and includes real-time visualization with evaluation on a standard dataset.

The framework has four components: data acquisition, algorithm, real-time visualization, and evaluation. Data is collected via phyphox app [10] (CSV export and HTTP live streaming). The StepCounter class maintains state and provides offline and update methods. A PyQt5 GUI [11] supports offline processing, live streaming, and ablation studies. An LSTM model [4] trained on OxWalk [5] serves as baseline.

The phyphox app [10] records linear acceleration, with optional gyroscope and magnetometer. Offline CSV files are loaded; online streaming polls the Remote Access API.

Raw tri-axial acceleration is rotated to world coordinates, then converted to magnitude. A cascade of filters is applied: causal moving average [2], Hampel outlier removal [3], Savitzky–Golay smoothing [8], and low-pass Butterworth [1].

The filtered magnitude signal goes to an adaptive peak detector. Minimum peak distance is set by autocorrelation (ACF) [2]. A refractory period prevents double counting; gyroscope and magnetometer energy gating reduces false positives. Streaming uses sliding windows with 50

The GUI displays real-time acceleration magnitude, step timestamps, cumulative counts, and step frequency. Evaluation mode runs parameter sets sequentially on test files, saving results.

LSTM [4] and bidirectional LSTM [9] models were trained. The 100 Hz OxWalk dataset [5] was resampled to 50 Hz, rotated to world coordinates, and windowed for step regression.

Evaluation on OxWalk [5] files showed that a baseline peak detector with low-pass filtering achieved less than 15%

mean relative error; the ACF-guided pipeline with full filtering reduced error to less than 10%. Extreme parameters caused overcounting, while disabling key filters led to undercounting. The LSTM underperformed compared to peak detection.

The project met its objectives: a stateful class with robust accelerometer-only operation, real-time visualization, and an ablation study confirming a balanced signal processing pipeline. It provides a solid baseline for IMU-based step counting.

2 Methodology

In this section, some key components are introduced. Section 2.1 introduces the DataPreprocessor class for extracting columns, receiving remote data, and returning formatted data chunks. Section 2.2 introduces the StepCounter class and the detailed algorithm implementation. Section 2.3 introduces how to plot figures and update real-time data in a demo using GUI. Section 2.4 introduces the pipeline of training LSTM [4, 9] models. Section 2.5 introduces the evaluation of different approaches through ablation tests.

2.1 Data Collection

Data is collected using phyphox [10]. In offline mode, several csv file(s) are fed into the DataPreprocessor class. The offline method in DataPreprocessor class receives a list, which can be a single file containing only the accelerometer data, or several files containing accelerometer, gyroscope or magnetometer data. Then the columns are extracted using pandas [7]. If additional IMU data besides acceleration is extracted, they will be linearly interpolated on accelerometer timestamps, which enables time consistency on the acceleration data. Then the data chunk can be fed into the StepCounter class in a strictly defined format.

In online mode, the polling logic on the instruction handbook is implemented. When calling the online method in DataPreprocessor class, IP and port parameters should be passed in. These are usually decided by phyphox [10] at the beginning. Considering that the buffer names can vary from device to device, the online method asks for it by requesting the \config to decide the buffer names before reading and writing operations when buffer_names parameter is not

passed in. Specifically, **yield** is used in the polling loop, so that data can be received from the generator and processed outside the loop.

The DataPreprocessor class does not do many things, and it is designed to extract columns, receive remote data, and return formatted data chunks only. The transformation, smoothing and windowing processes are done in the StepCounter class in order to keep consistency.

In practice, data sources are hand-recorded random walking and processed OxWalk [5] dataset (details see section 2.4). Some example data files are shown in the data_examples folder. The ground-truth of step counts is counted by myself in self-made dataset and summarization of the annotation column in processed OxWalk [5] dataset (details see section 2.4).

2.2 Algorithm

This is the most essential part in the whole project. Section 2.2.1 introduces how filters and rotation are applied. Section 2.2.2 introduces IMU fusion and ACF-guided methods. Section 2.2.3 and section 2.2.4 introduce implementations under two modes. Section 2.2.5 introduces the windowing strategy. Section 2.2.6 introduces the finite state machine method.

2.2.1 Filtering and Transformation. Several filters are applied in this project. First, a **low-pass filter** is applied to eliminate high-frequency noise from mobile sensors, which follows the Butterworth digital and analog filter design. The cutoff frequency is set to 1.0Hz, which is experiential. After several rounds of parameter tuning, this can be seen as an outstanding one. The main purpose is to maintain the indeed low-frequency signal as the step feature while removing disturbances. Also, the order of the filter is set to 3 for a better computational efficiency. The sampling rate is set to 50Hz globally, which is a trade-off between performance and efficiency.

$$y_i = \begin{cases} \frac{1}{i+1} \sum_{j=0}^i x_j & \text{if } i < L \\ \frac{1}{L} \sum_{j=i-L+1}^i x_j & \text{if } i \geq L \end{cases} \quad (1)$$

Second, a **causal moving average** as in Eq. (1) can be applied, where $L = 5$ is the window length and x, y is the signal value in the time series. To accelerate computation, a cumulative sum array is maintained for window summation.

Third, the **Hampel filter** and the **Savitzky-Golay filter** can be applied. The Hampel filter identifies and replaces outliers in the input data with the median value within a moving window, and the Savitzky-Golay filter can increase the precision of the data without distorting the signal tendency. The window size is set to 2 and the n_σ is set to 3.0 in the Hampel filter. The window size is set to 2 as well and the

order of the polynomial used to fit the samples is set to 3 in the Savitzky-Golay filter.

$$\mathbf{R} = \mathbf{I} + \sin \theta \cdot \mathbf{K} + (1 - \cos \theta) \cdot \mathbf{K}^2 \quad (2)$$

After smoothing, the coordinate transformation is applied with Rodrigues' formula as shown in Eq. (2), which converts sensor-frame acceleration into a world-frame where the vertical Z-axis aligns with gravity. Here $\mathbf{R}$ is the rotation matrix, and $\mathbf{a}_{\text{world}} = \mathbf{R} \cdot \mathbf{a}_{\text{sensor}}$ gives us the rotated acceleration. $\mathbf{I}$ is a 3 by 3 identity matrix, θ is the rotation angle, and $\mathbf{K}$ in Eq. (3) is a skew-symmetric matrix from $\mathbf{k} = \frac{\mathbf{v} \times \mathbf{u}}{\|\mathbf{v} \times \mathbf{u}\|}$, where $\mathbf{v}$ is the filtered and normalized sensor-frame acceleration, $\mathbf{u}$ is $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$, and $\theta = \arccos(\mathbf{v} \cdot \mathbf{u})$.

$$\mathbf{K} = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix} \quad (3)$$

Why rotation? The magnitude of acceleration is calculated later ensuring orientation robustness. This transformation, though, can isolate vertical motion in the Z-axis for orientation invariance, it is adapted to the LSTM [4, 9] training pipeline in section 2.4. There the training data is first rotated to the world coordinate to remove Z-axis gravity. Here keeps the consistency.

2.2.2 Adaptive Approaches. First, **ACF** is used to help extract step period. ACF can automatically estimate the main period within the current window (i.e., the time of one step) and then use this as a basis to set the distance parameter for peak detection, making the detection more aligned with the actual step frequency. In practice, the minimum distance between steps is set to 70% of dominant ACF period, and a constraint ranging from 0.5 to 2.5 steps per second is set during ACF calculation.

Second, **additional IMU signals** are taken into account if available. By utilizing the energy mean of the gyroscope and the energy variance of the magnetometer, the prominence and height thresholds for peak detection are dynamically adjusted, thereby reducing false detections during non-walking movements or stationary states, which is an energy gating strategy. The gyroscope is expected to generate adequate energy and adequate changes for the magnetometer to reach the threshold, otherwise a penalty is added to make peak detection harder. When penalty is accumulated to a upper bound, all peaks are ignored in this window.

Why adaptive? The primary objective is to enhance the accuracy and robustness of step detection, reducing false positives and false negatives caused by factors such as device placement (e.g., wrist or waist), changes in movement patterns (e.g., upstairs or downstairs), and environmental interference (e.g., phone on a table and shake the table).

2.2.3 Offline Mode. The offline mode receives data from a complete csv file. As described in Algo. 1, filters are applied to the complete data first, and then windowing is applied. The windowed acceleration magnitude and other optional IMU signals are processed in the `_process_window` method in the peak detection algorithm with an overlap of 50% window size. In contrast, no overlap for model prediction window and the estimated step counts are directly generated.

Algorithm 1: Run Offline

Input: {time(N), acc(N×3), gyro(N×3), mag(N×3)}
Output: {Step count S , step timestamps array T }

$S \leftarrow 0$, $T \leftarrow []$, $w_{\text{samples}} \leftarrow \lfloor w_{\text{sec}} \cdot f_s \rfloor$;
 $\text{acc}_{\text{world}} \leftarrow \text{FilterAndRotate}(\text{acc})$;
 $\text{mag}_{\text{acc}} \leftarrow \sqrt{\sum_{i=1}^3 \text{acc}_{\text{world}}[:, i]^2}$;
 $\text{start} \leftarrow 0$;
while $\text{start} + w_{\text{samples}} \leq N$ **do**
 $\text{end} \leftarrow \text{start} + w_{\text{samples}}$;
 $w_t \leftarrow \text{time}[\text{start} : \text{end}]$;
 $w_{\text{acc}} \leftarrow \text{mag}_{\text{acc}}[\text{start} : \text{end}]$;
 $w_{\text{gyro}} \leftarrow \text{gyro}[\text{start} : \text{end}]$;
 $w_{\text{mag}} \leftarrow \text{mag}[\text{start} : \text{end}]$;
 if $M \neq \text{None}$ **then**
 $\text{steps} \leftarrow \text{Predict}(\text{acc}[\text{start} : \text{end}])$;
 $S \leftarrow S + \text{steps}$;
 if $\text{steps} > 0$ **then**
 $t_{\frac{1}{2}} \leftarrow w_t[\lfloor w_{\text{samples}} \div 2 \rfloor]$;
 $T.\text{append}(t_{\frac{1}{2}})$;
 end
 $\text{start} \leftarrow \text{start} + w_{\text{samples}}$; // no overlap
 else
 if FSM **then**
 $\text{signal} \leftarrow w_{\text{mag}} - \mu(w_{\text{mag}})$;
 for $i \leftarrow 0$ **to** $w_{\text{samples}} - 1$ **do**
 $\text{step} \leftarrow \text{FSMUpdate}(w_t[i], \text{signal}[i])$;
 if step_detected **then**
 $S \leftarrow S + 1$; $T.\text{append}(w_t[i])$;
 end
 end
 $\text{start} \leftarrow \text{start} + w_{\text{samples}}$; // no overlap
 else
 $p \leftarrow \text{ProcessW}(w_t, w_{\text{acc}}, w_{\text{gyro}}, w_{\text{mag}})$;
 $S \leftarrow S + |p|$; $T.\text{extend}(p)$;
 $\text{start} \leftarrow \text{start} + \lfloor w_{\text{samples}} \div 2 \rfloor$; // 50%
 end
 end
end

2.2.4 Online Mode. The online mode in Algo. 2 receives and stores data chunks incrementally. Different from offline mode, online mode maintains temporary buffers. Data is filtered and rotated within the window. If the model is specified, step counts are generated directly. Otherwise, peak detection is called. Buffers are cleared completely when with a model and half when without a model.

Algorithm 2: Update

Input: {time(M), acc(M×3), gyro(M×3), mag(M×3)}
Output: {New steps ΔS , total steps S , timestamps T }

$\text{Buffer}_{\text{time}}.\text{extend}(\text{time})$; $\text{Buffer}_{\text{acc}}.\text{extend}(\text{acc})$;
if gyro exists **then**
 $\text{Buffer}_{\text{gyro}}.\text{extend}(\text{gyro})$
end
if mag exists **then**
 $\text{Buffer}_{\text{mag}}.\text{extend}(\text{mag})$
end
 $W_{\text{size}} \leftarrow \lfloor T_w \times f_s \rfloor$, $P_{\text{total}} \leftarrow \emptyset$, $S_M \leftarrow 0$;
while $\text{length}(\text{Buffer}_{\text{time}}) \geq W_{\text{size}}$ **do**
 $w_t \leftarrow \text{Buffer}_{\text{time}}[: W_{\text{size}}]$;
 $w_{\text{acc}} \leftarrow \text{Buffer}_{\text{acc}}[: W_{\text{size}}]$;
 $w_{\text{gyro}} \leftarrow \text{Buffer}_{\text{gyro}}[: W_{\text{size}}]$;
 $w_{\text{mag}} \leftarrow \text{Buffer}_{\text{mag}}[: W_{\text{size}}]$;
 $\text{acc}_{\text{world}} \leftarrow \text{FilterAndRotate}(w_{\text{acc}})$;
 $\text{mag}_{\text{acc}} \leftarrow \sqrt{\sum_{i=1}^3 \text{acc}_{\text{world}}[:, i]^2}$;
 if M **then**
 $S_M \leftarrow S_M + \text{Predict}(w_{\text{acc}})$;
 $\text{AllBuffers} \leftarrow \text{AllBuffers}[W_{\text{size}} :]$;
 else
 if FSM **then**
 $\text{signal} \leftarrow \text{mag}_{\text{acc}} - \mu(\text{mag}_{\text{acc}})$;
 for $i \leftarrow 0$ **to** $W_{\text{size}} - 1$ **do**
 $\text{step} \leftarrow \text{FSMUpdate}(w_t[i], \text{signal}[i])$;
 if step **then**
 $P_{\text{total}}.\text{append}(w_t[i])$;
 end
 end
 $\text{AllBuffers} \leftarrow \text{AllBuffers}[W_{\text{size}} :]$;
 else
 $p \leftarrow \text{ProcessW}(w_t, \text{mag}_{\text{acc}}, w_{\text{gyro}}, w_{\text{mag}})$;
 $P_{\text{total}}.\text{extend}(p)$;
 $\text{AllBuffers} \leftarrow \text{AllBuffers}[\lfloor W_{\text{size}} \div 2 \rfloor :]$;
 end
 end
end
end
 $S \leftarrow S + (M ? S_M : \text{length}(P_{\text{total}}))$; $T.\text{extend}(P_{\text{total}})$;

2.2.5 Windowing. Given a long time series, it's hard to directly extract step counts, so windowing is necessary. The offline mode filters first, then splits into windows, while the online mode does everything in the window. The window size in this project is decided by both a window period and the sampling rate, which is $W_{size} \leftarrow \lfloor T_w \times f_s \rfloor$.

Algo. 3 shows what happens in a window. Several noise filtering approaches are applied, e.g., peaks are ignored when the standard deviation is under the threshold or the accumulated penalty is above the threshold. The penalty is applied to the prominence and height parameters in the `find_peaks` method. ACF is utilized to estimate the step period in a window and to influence the distance parameter in the `find_peaks` method. Since windows have overlap between each other, a refractory timer is set to 0.75 seconds to avoid double counting.

Why windowing? The primary purpose is to enhance the temporal resolution and robustness of gait detection, preventing the loss or missed detection of key gait features due to window boundary cutting. Overlap increases the probability that each gait event will fall entirely within a single window.

2.2.6 Finite State Machine. Simple finite state machine is applied as an additional method. In practice, the FSM has 3 states and 5 transitions, as shown in Fig. 1. One step is counted when one cycle of Peak $\rightarrow$ ZeroCrossing $\rightarrow$ Valley is finished. The FSM is not complex, but it achieves commensurate results as the peak detection method.

2.3 Visualization

A live demo implemented by PyQt5 [11] is the visualization part. Raw signals and filtered signals are shown. Online mode can display updated data in real time. The red lines are timestamps that steps detected. This part is mainly done by AI.

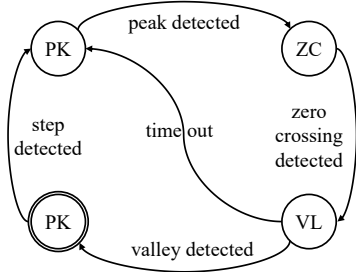


Figure 1: The finite state machine.

Algorithm 3: Process Window

Input: $w_t, w_{acc}, w_{gyro}, w_{mag}$
Output: Peak timestamps P , ACF period T

```

 $\mu \leftarrow \text{mean}(w_{acc}), \sigma \leftarrow \text{std}(w_{acc});$ 
if  $\sigma < \theta_{noise}$  then
  | return  $\emptyset, 0;$  // noise filtering
end
 $P_{dyn} \leftarrow 0.5, H_{pen} \leftarrow 0.0;$ 
if  $w_{gyro} \neq \emptyset$  and  $\text{mean}(|w_{gyro}|^2) < E_{min}^{gyro}$  then
  |  $P_{dyn} \leftarrow P_{dyn} + P_{penalty}, H_{pen} \leftarrow H_{pen} + P_{penalty};$ 
end
if  $w_{mag} \neq \emptyset$  and  $\text{var}(|w_{mag}|) < V_{min}^{mag}$  then
  |  $P_{dyn} \leftarrow P_{dyn} + P_{penalty}, H_{pen} \leftarrow H_{pen} + P_{penalty};$ 
end
if  $P_{dyn} > 1.0$  then
  | return  $\emptyset, 0$ 
end
 $T \leftarrow \text{ACF\_Period}(w_{acc});$  // autocorrelation
 $D_{min} \leftarrow \max(\lfloor T \times f_s \times W_{acf} \rfloor, 1);$  // dynamic dis
 $H_{thresh} \leftarrow \max(\mu + \alpha \cdot \sigma, \mu + \theta_{abs}) + H_{pen};$ 
// adaptive height threshold
 $P_{idx} \leftarrow \text{FindPeaks}(w_{acc}, \text{height} =$ 
 $H_{thresh}, \text{prominence} = P_{dyn}, \text{distance} = D_{min});$ 
 $P_{valid} \leftarrow \emptyset;$ 
foreach  $i \in P_{idx}$  do
  |  $t_{peak} \leftarrow w_t[i];$ 
  | if  $t_{peak} - t_{last\_peak} > T_{refractory}$  then
    | |  $P \leftarrow P \cup \{t_{peak}\}; t_{last\_peak} \leftarrow t_{peak};$ 
  | end
end
return  $P, T$ 
  
```



Figure 2: The demo. The figure shows offline (upper) and online (lower) mode.

2.4 Model Training

An LSTM [4] model and a bidirectional LSTM [9] model are trained as the machine learning approach.

First, the hip and wrist at 100 Hz part of OxWalk [5] dataset is picked. Second, a low-pass filter with a cutoff frequency of 1 Hz is applied. Third, the data is rotated to the world frame and the gravity is subtracted from the corresponding axis (IDK why but it's the x-axis). Fourth, a high-pass filter with a cutoff frequency of 0.5 Hz is applied. Fifth, the 100 Hz sampling rate dataset is resampled to 50 Hz according to the algorithms in StepCounter. Sixth, the data chunks are windowed and annotated. Seventh, the preprocessed data is feed into a 5-fold cross validation and finally, a sequential model is trained using an Adam [6] optimizer.

2.5 Evaluation

The evaluation dataset samples 29 files from the preprocessed dataset from 2.4. The baseline is selected as the peak detection algorithm with only low-pass filter. Then all filters and the ACF guidance are turned on. Some components are turned off and some parameters are tuned for ablation tests. The model and FSM are also evaluated. The metrics here are MAE and mean relative error. An evaluation view is included in the demo, allowing users to add tasks conveniently.

method	MAE	error rate
baseline	66.00	6.77%
all filters+ACF	74.79	6.98%
w/o ACF	72.31	6.85%
w/o filters except LPF	67.90	6.85%
LSTM	538.48	55.81%
FSM	178.10	15.91%

Table 1: Ablation test results.

The result from Tbl. 1 shows that the baseline achieves a comparative acceptable error rate. The advanced methods do not seem to work, which is confusing.

parameters		MAE	error rate
window_sec	1.0	120.59	13.54%
	4.0	85.66	8.60%
refractory_sec	0.1	64.90	7.14%
	1.0	150.62	14.30%
cutoff_high	1.0	566.38	59.35%
	4.0	187.00	26.82%
sampling rate	25.0	207.41	26.32%
	75.0	247.14	26.77%

Table 2: Experiment results.

The result from Tbl. 2 shows that extending window length or shortening refractory time can comparatively lower error rates, while modifying other parameters does not seem to have a good effect.

3 Limitations

The algorithm's generalization ability is not so good, and it can't handle some border cases, e.g., when shaking the phone violently and generating a lot of high-frequency peaks, but actually no step is generated. **I sincerely hope the test performance on other datasets does not hurt the grading so much. :)**

The IMU fusion evaluation is not sufficient with the lack of data. Tested on my hand-recorded data, the results keep the same w/o other IMU signals.

The finite state machine is too simple to model a more accurate process. Also, the data is processed per window, which could hurt consistency. But without doing so, the data cannot oscillate around 0, which makes zero crossing fail.

References

- [1] S. Butterworth. 1930. On the Theory of Filter Amplifiers. *Experimental Wireless & the Wireless Engineer* 7 (Oct. 1930), 536–541.
- [2] James D. Hamilton. 1994. *Time Series Analysis*. Princeton University Press, Princeton, NJ. 816 pages.
- [3] Frank R. Hampel. 1974. The Influence Curve and Its Role in Robust Estimation. *J. Amer. Statist. Assoc.* 69, 346 (1974), 383–393. <http://www.jstor.org/stable/2285666>
- [4] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (11 1997), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735> arXiv:<https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf>
- [5] S Khalid, S R Small, A Doherty, L Von Fritsch, and A Price. 2022. OxWalk: Wrist and hip-based activity tracker dataset for free-living step detection and gait recognition. <https://doi.org/10.5287/BODLEIAN:ORQ2ABNBR>
- [6] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG] <https://arxiv.org/abs/1412.6980>
- [7] The pandas development team. 2020. *pandas-dev/pandas: Pandas*. <https://doi.org/10.5281/zenodo.3509134>
- [8] Abraham. Savitzky and M. J. E. Golay. 1964. Smoothing and Differentiation of Data by Simplified Least Squares Procedures. *Analytical Chemistry* 36, 8 (1964), 1627–1639. <https://doi.org/10.1021/ac60214a047> arXiv:<https://doi.org/10.1021/ac60214a047>
- [9] M. Schuster and K.K. Paliwal. 1997. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing* 45, 11 (1997), 2673–2681. <https://doi.org/10.1109/78.650093>
- [10] S Staacks, S Hütz, H Heinke, and C Stampfer. 2018. Advanced tools for smartphone-based experiments: phyphox. *Physics Education* 53, 4 (may 2018), 045009. <https://doi.org/10.1088/1361-6552/aac05e>
- [11] The Qt Company. 2026. Qt | Tools for Each Stage of Software Development Lifecycle. <https://www.qt.io/>. Accessed: 2026-03-24.